

Module 8: Security and Reliability

Guardrails for AI-assisted systems

Module 8: Security and Reliability

Primary sources: Osmani, Chapter 8; Thomas & Hunt, Topics 15, 40, and 43.

Learning Outcomes

- Identify security risks in AI-generated code.
 - Analyze reliability challenges.
 - Evaluate safety, performance, and complexity tradeoffs.
 - Propose strategies to improve robustness.
-

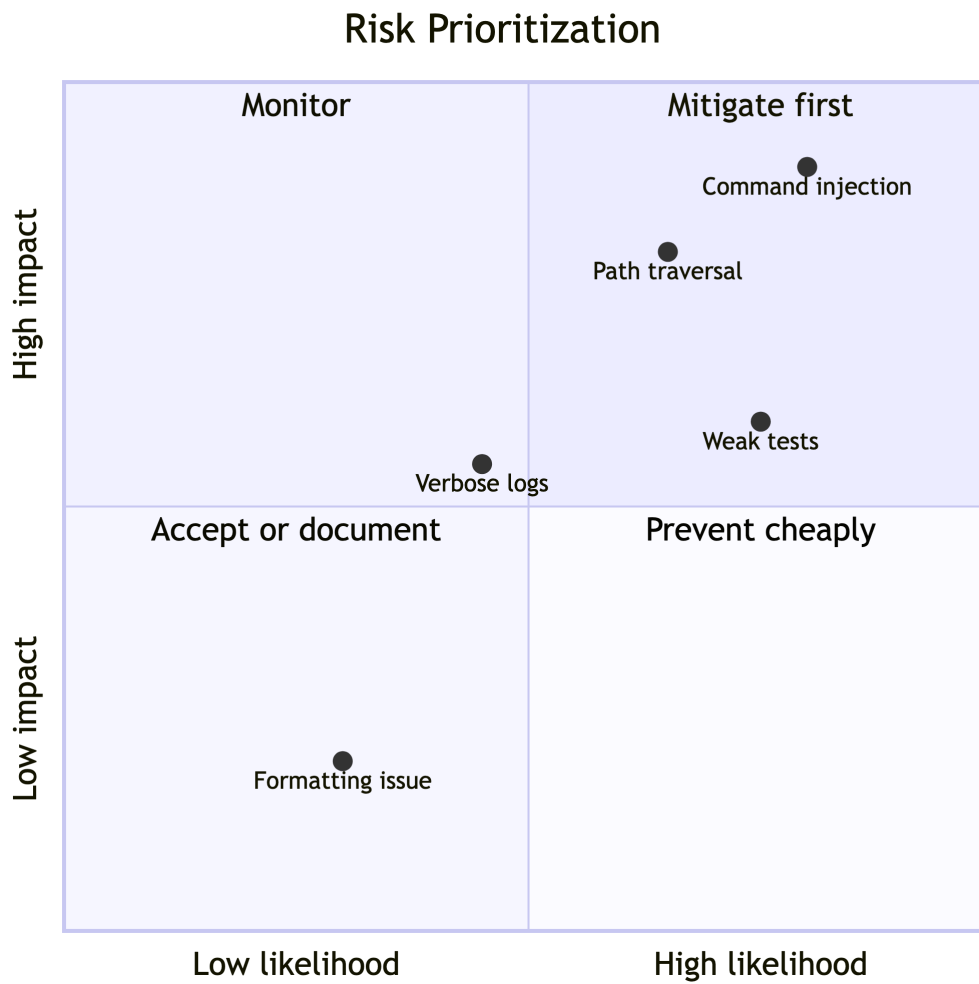
AI-Specific Risk Pattern

AI often generates code optimized for the visible task, not the threat model.

Risks include:

- Missing authorization checks.
 - Insecure defaults.
 - Unsafe command execution.
 - Overbroad file access.
 - Dependency confusion.
 - Logging secrets.
-

Risk Matrix



Security Review Areas

- Input validation.
- Authentication and authorization.
- Secrets handling.
- Dependency versions.
- Path traversal.
- Command injection.
- Network access.

- Audit logs.
-

AI Security Audit Loop

Osmani recommends layering review rather than trusting a single pass.

- Run automated scanners and dependency audits.
- Optionally ask a separate AI or review mode to critique security assumptions.
- Use a human checklist for sensitive flows.
- Add security-focused tests for malicious inputs and unauthorized access.
- Slow down when generated code touches credentials, files, commands, or user data.

AI critique is not a security control by itself. It can supplement, not replace, threat modeling, code review, tests, and scanners.

Reliability Review Areas

- Timeouts.
 - Retries and backoff.
 - Deterministic test paths.
 - Clear failure states.
 - Idempotent operations.
 - Recoverable artifacts.
 - Observability.
-

Reliability Needs Evidence

AI assistance does not lower reliability requirements.

- Put tests and security scans in CI.
 - Include failure-path tests, not just happy paths.
 - Check performance with measurement before optimizing.
 - Use logging that helps diagnosis without leaking sensitive data.
 - Monitor production behavior so reliability claims can be checked.
-

Refactoring AI Code

Refactoring turns generated code into maintainable code without changing behavior.

Use tests first, then improve:

- Names.
- Duplication.
- Boundaries.
- Error handling.
- Configuration.

Thomas and Hunt's guardrails: do not refactor and add functionality at the same time, keep steps small, and run tests after each step.

Estimating Risk

Estimating is about building a model of uncertainty, not guessing a single number.

For AI-assisted reliability work, estimate:

- Scope: what security or reliability question is being asked?
 - Drivers: which inputs, dependencies, or usage patterns dominate risk?
 - Range: optimistic, likely, and pessimistic outcomes.
 - Mitigation: what concrete action reduces uncertainty?
 - Evidence: what test, review, log, or policy supports the estimate?
-

Safety Tradeoffs

Robustness is not free.

- More validation can improve safety but add complexity.
- More logging can improve diagnosis but risk secret exposure.
- More retries can improve reliability but amplify load.
- More abstraction can improve maintainability but obscure behavior.
- More security checks can slow development but prevent expensive failures.

Make the tradeoff explicit and document the reason.

Safeguard Examples

- Reject paths outside workspace.
 - Require confirmation before writes.
 - Disable shell commands by default.
 - Redact environment variables from logs.
 - Add tests for malicious inputs.
 - Fail closed when model output is malformed.
-

Lab 4 Final Connection

Your submission should include:

- Specific failure modes.
 - Risk ratings.
 - Concrete mitigations.
 - At least two safeguards or safety tests.
 - Production readiness plan.
 - Copilot-assisted analysis corrections.
-

Practice Activity

Threat-model your Java agent:

- What input could cause harm?
- What tool could be misused?
- What output could mislead a user?
- What safeguard would fail safely?

Produce a four-row risk matrix with likelihood, impact, mitigation, and evidence for each risk.

Takeaways

- Generated code is not security-reviewed code.
- Reliability requires explicit design for failure.
- Safeguards must be implemented and tested, not merely described.